

막차 카멜레온

Vite + React + JavaScript 작업 가이드

대상: Vite + React + JavaScript와 컴포넌트 기본을 배운 상태에서, PDF 기획안을 보며 개인 프로젝트로 코딩하는 제작자

이 문서의 목적

- TypeScript, Redux, 게임 엔진 없이 Vite + React + JavaScript + Canvas API만으로 MVP를 제작한다.
- 작업 순서를 지키면서 “한 판이 돌아가는 구조”를 먼저 만들고, 그 뒤 위장/그림 기능을 붙인다.
- React는 화면과 버튼, Canvas는 게임 화면을 담당한다는 기준을 끝까지 유지한다.
- 각 단계마다 파일, 할 일, 완료 기준, 자주 터지는 실수를 확인하면서 진행한다.

절대 기준

React는 UI, Canvas는 게임. 먼저 5층에서 도형이 움직이고 술레가 층을 바꿔 클릭으로 찾는 뼈대를 만든다. 그 다음 색상, 스포이드, 확대 편집, 힌트박스, 조준 커서, 흔적을 붙인다.

구분	이번 MVP에서 쓸 것	이번 MVP에서 쓰지 않을 것
기술	Vite, React, JavaScript, CSS, Canvas API	TypeScript, Redux, Zustand, React Router, Phaser, PixiJS
개발 방식	작은 단계별 완성, 기능마다 완료 기준 확인	처음부터 전체 기능을 한 번에 구현
게임 구조	로컬 테스트 중심, 한 화면에서 상태 전환	온라인 멀티플레이, 로그인, 랭킹

0. 작업 전에 준비할 기준 문서

기존 MVP 기획안은 “무엇을 만들지”의 기준이고, 이 문서는 “어떤 순서로 어떻게 만들지”의 기준이다. 코딩 중에는 두 문서를 같이 열어둔다.

문서/자료	역할	작업 중 볼 때
MVP 기획안 PDF	게임 규칙, 5층 구조, 화면 레이아웃, 제외할 기능 확인	기능을 추가하고 싶을 때 MVP 범위인지 판단
이 작업 가이드 PDF	작업 순서, 파일 구조, 구현 체크리스트 확인	오늘 무엇을 만들지 결정
실행 중인 브라우저	현재 동작 확인	기능 하나 만들 때마다 테스트
개발자 도구 Console	에러 확인	화면이 멈추거나 클릭/키보드가 안 될 때

이번 프로젝트의 3대 원칙

- 1 화면 전환과 HUD는 React 컴포넌트로 만든다.
- 2 프레임마다 움직이는 좌표, 키 입력, 마우스 좌표는 useRef 안의 객체로 관리한다.
- 3 useState는 화면에 표시해야 하는 값만 관리한다. 예: 현재 화면, 클릭 수, 승패, 선택 도형, 모달 열림.

왜 이렇게 나누나?

도형 x/y 좌표를 useState로 매 프레임 갱신하면 React가 계속 리렌더링된다. 초반에는 돌아갈 수 있지만, 흔적/그림/조준 커서가 붙으면 느려지고 구조가 꼬인다. 자주 바뀌는 게임 값은 useRef에 넣고, Canvas는 requestAnimationFrame으로 직접 그린다.

1. 프로젝트 세팅

목표

Vite + React + JavaScript 프로젝트를 만들고, 기본 화면이 실행되는 상태까지 간다.

터미널 명령어

```
npm create vite@latest last-train-chameleon -- --template react
cd last-train-chameleon
npm install
npm run dev
```

처음 정리할 파일

```
src/
  App.jsx
  main.jsx
  App.css
  components/
  game/
```

기본 폴더 만들기

```
src/
  components/
    StartScreen.jsx
    ShapeSelectScreen.jsx
    GameScreen.jsx
    ResultScreen.jsx
  game/
    floors.js
    draw.js
    collision.js
```

완료 기준

- npm run dev 실행 시 브라우저가 열린다.
- App.jsx 수정 시 화면이 바로 갱신된다.
- components와 game 폴더를 만들어두었다.

실수 방지

- 파일 이름 대소문자를 섞지 않는다. 예: GameScreen.jsx로 만들었으면 import도 정확히 GameScreen.jsx 기준으로 한다.
- 처음부터 Tailwind, Router, 상태관리 라이브러리를 설치하지 않는다.
- 프로젝트명은 영어 폴더명으로 둔다. 한글/공백 폴더명은 나중에 경로 문제를 만들 수 있다.

2. 화면 상태 전환 만들기

목표

StartScreen -> ShapeSelectScreen -> GameScreen -> ResultScreen 흐름을 만든다. 아직 Canvas 게임을 만들지 않는다.

App.jsx 기본 구조

```
import { useState } from 'react';
import './App.css';
import StartScreen from './components/StartScreen';
import ShapeSelectScreen from './components/ShapeSelectScreen';
import GameScreen from './components/GameScreen';
import ResultScreen from './components/ResultScreen';

function App() {
  const [screen, setScreen] = useState('start');
  const [selectedShape, setSelectedShape] = useState(null);
  const [winner, setWinner] = useState(null);

  return (
    <>
      {screen === 'start' && (
        <StartScreen onStart={() => setScreen('shapeSelect')} />
      )}

      {screen === 'shapeSelect' && (
        <ShapeSelectScreen
          onSelect={(shape) => {
            setSelectedShape(shape);
            setScreen('game');
          }}
        />
      )}

      {screen === 'game' && (
        <GameScreen
          selectedShape={selectedShape}
          onGameEnd={(result) => {
            setWinner(result);
            setScreen('result');
          }}
        />
      )}

      {screen === 'result' && (
        <ResultScreen winner={winner} onRestart={() => setScreen('start')} />
      )}
    </>
  );
}

export default App;
```

컴포넌트 역할

컴포넌트	해야 할 일	아직 하지 말 것
StartScreen	게임명, 설명, 게임 시작 버튼	게임 로직
ShapeSelectScreen	원/세모/네모/별 버튼	도형 그리기
GameScreen	Canvas 자리, 임시 종료 버튼	전체 게임 기능 한 번에 구현
ResultScreen	승자 표시, 다시 하기 버튼	랭킹/저장

완료 기준

- 게임 시작 버튼을 누르면 도형 선택 화면으로 간다.
- 도형을 누르면 게임 화면으로 간다.
- 게임 화면에서 임시 버튼으로 결과 화면으로 갈 수 있다.
- 결과 화면에서 다시 시작할 수 있다.

3. Canvas와 게임 루프 만들기

목표

GameScreen 안에 Canvas를 띄우고, requestAnimationFrame으로 계속 다시 그린다. 이 단계에서는 배경만 그려도 된다.

GameScreen.jsx 뼈대

```
import { useEffect, useRef } from 'react';

function GameScreen({ selectedShape, onGameEnd }) {
  const canvasRef = useRef(null);
  const frameRef = useRef(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    const ctx = canvas.getContext('2d');

    function loop() {
      ctx.clearRect(0, 0, canvas.width, canvas.height);

      // 1. 배경
      ctx.fillStyle = '#e8f5ed';
      ctx.fillRect(0, 0, canvas.width, canvas.height);

      // 2. 게임 캔버스
      ctx.fillStyle = '#0e6b4d';
      ctx.font = '24px sans-serif';
      ctx.fillText('Game Canvas', 40, 50);

      frameRef.current = requestAnimationFrame(loop);
    }

    loop();

    return () => cancelAnimationFrame(frameRef.current);
  }, []);

  return (
    <div className="game-screen">
      <canvas ref={canvasRef} width={800} height={500} />
      <button onClick={() => onGameEnd('hider')}>숨기 버튼</button>
    </div>
  );
}

export default GameScreen;
```

완료 기준

- Canvas가 보인다.
- 배경색이 칠해진다.
- 임시 텍스트가 보인다.
- 화면 전환 후에도 에러가 없다.

자주 터지는 실수

- canvasRef.current가 null인데 getContext를 호출하는 경우: useEffect 안에서 처리한다.
- requestAnimationFrame 정리를 안 하는 경우: return에서 cancelAnimationFrame을 한다.
- Canvas width/height를 CSS만으로 키우는 경우: 실제 좌표가 틀어진다. width/height 속성도 지정한다.

4. 5층 데이터와 배경 렌더링

목표

1F~5F 데이터를 만들고, 현재 층에 따라 다른 배경을 Canvas에 그린다.

game/floors.js

```
export const floors = [
  { id: 1, name: '검문실', theme: 'check' },
  { id: 2, name: '벽길', theme: 'brick' },
  { id: 3, name: '창고', theme: 'storage' },
  { id: 4, name: '숲길', theme: 'forest' },
  { id: 5, name: '야간', theme: 'night' },
];

export function getFloorById(id) {
  return floors.find((floor) => floor.id === id);
}
```

game/draw.js - 배경 함수

```
export function drawFloorBackground(ctx, floor, width, height) {
  if (floor.theme === 'check') drawCheckRoom(ctx, width, height);
  if (floor.theme === 'brick') drawBrickAlley(ctx, width, height);
  if (floor.theme === 'storage') drawStorageRoom(ctx, width, height);
  if (floor.theme === 'forest') drawForest(ctx, width, height);
  if (floor.theme === 'night') drawNightFestival(ctx, width, height);
}

function drawCheckRoom(ctx, width, height) {
  ctx.fillStyle = '#eeeeee';
  ctx.fillRect(0, 0, width, height);

  const size = 50;
  for (let y = 0; y < height; y += size) {
    for (let x = 0; x < width; x += size) {
      ctx.fillStyle = (x / size + y / size) % 2 === 0 ? '#d8d8d8' : '#f7f7f7';
      ctx.fillRect(x, y, size, size);
    }
  }
}
```

GameScreen에서 currentFloor 상태 추가

```
const [currentFloor, setCurrentFloor] = useState(1);
const currentFloorData = getFloorById(currentFloor);
```

완료 기준

- 1F~5F 버튼을 누르면 currentFloor가 바뀐다.
- currentFloor에 따라 Canvas 배경이 달라진다.
- 각 층의 이름이 HUD에 표시된다.

5. 도형 선택과 도형 렌더링

목표

도형 선택 화면에서 고른 원/세모/네모/별을 Canvas에 그린다.

도형 상태는 useRef에 둔다

```
const playerRef = useRef({
  x: 380,
  y: 250,
  floor: 1,
  size: 52,
  speed: 3,
  color: '#2ecc71',
});
```

drawShape 함수

```
export function drawShape(ctx, shape, player) {
  ctx.save();
  ctx.fillStyle = player.color;
  ctx.strokeStyle = 'rgba(0, 0, 0, 0.25)';
  ctx.lineWidth = 2;

  if (shape === 'circle') drawCircle(ctx, player);
  if (shape === 'square') drawSquare(ctx, player);
  if (shape === 'triangle') drawTriangle(ctx, player);
  if (shape === 'star') drawStar(ctx, player);

  ctx.fill();
  ctx.stroke();
  ctx.restore();
}

function drawCircle(ctx, p) {
  ctx.beginPath();
  ctx.arc(p.x, p.y, p.size / 2, 0, Math.PI * 2);
}

function drawSquare(ctx, p) {
  ctx.beginPath();
  ctx.rect(p.x - p.size / 2, p.y - p.size / 2, p.size, p.size);
}

function drawTriangle(ctx, p) {
  const r = p.size / 2;
  ctx.beginPath();
  ctx.moveTo(p.x, p.y - r);
  ctx.lineTo(p.x - r, p.y + r);
  ctx.lineTo(p.x + r, p.y + r);
  ctx.closePath();
}
```

완료 기준

- 원 선택 시 원이 나온다.
- 세모/네모/별도 선택에 따라 나온다.
- 아직 이동하지 않아도 된다.
- 현재 감시층과 숨는 사람 층이 같을 때만 도형을 보이게 할 준비를 한다.

6. 키보드 이동 구현

목표

WASD 또는 방향키로 도형을 움직인다. 위치는 useRef 안의 playerRef.current.x/y를 직접 바꾼다.

키 입력 저장

```
const keysRef = useRef({});

useEffect(() => {
  function handleKeyDown(e) {
    keysRef.current[e.key] = true;
  }

  function handleKeyUp(e) {
    keysRef.current[e.key] = false;
  }

  window.addEventListener('keydown', handleKeyDown);
  window.addEventListener('keyup', handleKeyUp);

  return () => {
    window.removeEventListener('keydown', handleKeyDown);
    window.removeEventListener('keyup', handleKeyUp);
  };
}, []);
```

loop 안에서 이동 처리

```
function updatePlayer() {
  const player = playerRef.current;
  const keys = keysRef.current;

  if (keys.ArrowLeft || keys.a) player.x -= player.speed;
  if (keys.ArrowRight || keys.d) player.x += player.speed;
  if (keys.ArrowUp || keys.w) player.y -= player.speed;
  if (keys.ArrowDown || keys.s) player.y += player.speed;

  const half = player.size / 2;
  player.x = Math.max(half, Math.min(canvas.width - half, player.x));
  player.y = Math.max(half, Math.min(canvas.height - half, player.y));
}
```

완료 기준

- 도형이 WASD/방향키로 움직인다.
- 도형이 Canvas 밖으로 나가지 않는다.
- 움직일 때 React 화면 전체가 깜빡이지 않는다.
- 키를 떼면 멈춘다.

주의

player.x를 useState로 만들지 않는다. 지금부터 자주 바뀌는 게임 값은 playerRef.current에 둔다.

7. 계단과 엘리베이터 이동

목표

숨은 사람은 층 버튼으로 이동하지 못한다. 계단 또는 엘리베이터 위치까지 직접 이동해야 층을 바꿀 수 있다.

층별 이동 지점 데이터

```
export const floorObjects = {
  1: { stairs: [{ x: 70, y: 420, to: 2 }], elevator: { x: 720, y: 420 } },
  2: { stairs: [{ x: 720, y: 420, to: 1 }, { x: 70, y: 420, to: 3 }], elevator: { x: 400, y: 420 } },
  3: { stairs: [{ x: 70, y: 420, to: 2 }, { x: 720, y: 420, to: 4 }], elevator: { x: 400, y: 420 } },
  4: { stairs: [{ x: 720, y: 420, to: 3 }, { x: 70, y: 420, to: 5 }], elevator: { x: 400, y: 420 } },
  5: { stairs: [{ x: 720, y: 420, to: 4 }], elevator: { x: 70, y: 420 } },
};
```

근처 판정 함수

```
function isNear(a, b, range = 45) {
  const dx = a.x - b.x;
  const dy = a.y - b.y;
  return Math.sqrt(dx * dx + dy * dy) < range;
}
```

E키로 계단 사용

```
function tryUseStairs() {
  const player = playerRef.current;
  const objects = floorObjects[player.floor];

  const nearStair = objects.stairs.find((stair) => isNear(player, stair));
  if (!nearStair) return;

  player.floor = nearStair.to;
  player.x = 400;
  player.y = 250;
  setCurrentFloor(player.floor); // HUD/□□ □□□
}
```

완료 기준

- 계단 위치가 Canvas에 보인다.
- 계단 근처에서 E키를 누르면 인접 층으로 이동한다.
- 층 이동 후 배경이 바뀐다.
- 엘리베이터는 이 단계에서 버튼만 띄워도 되고, 실제 원하는 층 이동은 다음 단계로 미뤄도 된다.

8. 술레 감시층 전환과 클릭 판정

목표

술레는 1F~5F 버튼으로 감시층을 빠르게 바꾸고, 현재 감시층에서 도형을 클릭해 찾는다.

상태

```
const [watchedFloor, setWatchedFloor] = useState(1);
const [clicksLeft, setClicksLeft] = useState(15);
```

층 버튼 UI

```
<div className="floor-tabs">
  {[1, 2, 3, 4, 5].map((floor) => (
    <button
      key={floor}
      className={watchedFloor === floor ? 'active' : ''}
      onClick={() => setWatchedFloor(floor)}
    >
      {floor}F
    </button>
  ))}
</div>
```

Canvas 클릭 좌표 구하기

```
function getMousePos(canvas, event) {
  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;

  return {
    x: (event.clientX - rect.left) * scaleX,
    y: (event.clientY - rect.top) * scaleY,
  };
}
```

클릭 판정

```
function handleCanvasClick(e) {
  const canvas = canvasRef.current;
  const mouse = getMousePos(canvas, e);
  const player = playerRef.current;

  if (watchedFloor !== player.floor) {
    setClicksLeft((prev) => prev - 1);
    return;
  }

  const dx = mouse.x - player.x;
  const dy = mouse.y - player.y;
  const distance = Math.sqrt(dx * dx + dy * dy);

  if (distance < player.size / 2) {
    onGameEnd('seeker');
  } else {
    setClicksLeft((prev) => prev - 1);
  }
}
```

완료 기준

- 술레가 층 버튼을 누르면 감시층이 바뀐다.
- 감시층과 숨는 사람 층이 같고, 도형 근처를 클릭하면 술레 승리.
- 틀리면 클릭 수가 줄어든다.

- 클릭 수가 0이 되면 숨는 사람 승리로 연결할 준비가 된다.

9. 타이머와 승패 처리

목표

120초 동안 숨는 사람이 버티면 숨는 사람 승리. 술레가 클릭 성공하면 술레 승리. 클릭 수가 0이 되면 숨는 사람 승리.

시간 상태

```
const [timeLeft, setTimeLeft] = useState(120);

useEffect(() => {
  const timerId = setInterval(() => {
    setTimeLeft((prev) => prev - 1);
  }, 1000);

  return () => clearInterval(timerId);
}, []);

useEffect(() => {
  if (timeLeft <= 0) {
    onGameEnd('hider');
  }
}, [timeLeft, onGameEnd]);

useEffect(() => {
  if (clicksLeft <= 0) {
    onGameEnd('hider');
  }
}, [clicksLeft, onGameEnd]);
```

완료 기준

- 시간이 1초마다 줄어든다.
- 0초가 되면 결과 화면으로 간다.
- 클릭 수가 0이면 결과 화면으로 간다.
- 술레가 정답 클릭하면 즉시 결과 화면으로 간다.

주의

- 결과 화면으로 넘어간 뒤에도 타이머가 계속 도는 버그를 확인한다.
- onGameEnd가 여러 번 호출되어 화면이 이상하게 바뀌지 않는지 확인한다.
- 개발 중에는 시간을 10초로 줄여 빠르게 테스트한다.

10. 술레 조준 커서와 힌트박스

목표

술레 마우스를 따라다니는 조준 그래픽과, 술레 화면 하단의 힌트박스를 만든다. 둘 다 아이템이 아니라 기본 UI다.

마우스 좌표 저장

```
const mouseRef = useRef({ x: 0, y: 0, inside: false });

function handleMouseMove(e) {
  const canvas = canvasRef.current;
  mouseRef.current = { ...getMousePos(canvas, e), inside: true };
}

function handleMouseLeave() {
  mouseRef.current.inside = false;
}
```

Canvas에 조준점 그리기

```
function drawCrosshair(ctx, mouse) {
  if (!mouse.inside) return;

  ctx.save();
  ctx.strokeStyle = 'rgba(20, 20, 20, 0.8)';
  ctx.lineWidth = 2;

  ctx.beginPath();
  ctx.arc(mouse.x, mouse.y, 12, 0, Math.PI * 2);
  ctx.moveTo(mouse.x - 22, mouse.y);
  ctx.lineTo(mouse.x - 8, mouse.y);
  ctx.moveTo(mouse.x + 8, mouse.y);
  ctx.lineTo(mouse.x + 22, mouse.y);
  ctx.moveTo(mouse.x, mouse.y - 22);
  ctx.lineTo(mouse.x, mouse.y - 8);
  ctx.moveTo(mouse.x, mouse.y + 8);
  ctx.lineTo(mouse.x, mouse.y + 22);
  ctx.stroke();

  ctx.restore();
}
```

힌트박스 상태

```
const [hint, setHint] = useState({
  current: '□□',
  history: [],
});

function updateHint(newHint) {
  setHint((prev) => ({
    current: newHint,
    history: prev.current ? [prev.current, ...prev.history].slice(0, 2) : prev.history,
  }));
}
```

완료 기준

- 술레 화면에서 마우스를 움직이면 조준점이 따라다닌다.
- Canvas 밖으로 나가면 조준점이 사라진다.
- 힌트박스에 현재 컨셉과 이전 컨셉이 표시된다.
- 힌트 텍스트는 맵 위에 절대 표시하지 않는다.

11. 색상 팔레트와 스포이드

목표

숨는 사람이 도형의 색을 바꿀 수 있고, Canvas 배경의 픽셀 색을 찍어서 사용할 수 있게 한다.

색상 팔레트 상태

```
const [selectedColor, setSelectedColor] = useState('#2ecc71');
const palette = ['#222222', '#ffffff', '#e74c3c', '#f1c40f', '#2ecc71', '#3498db',
  '#8e44ad', '#8b5a2b'];

function changePlayerColor(color) {
  setSelectedColor(color);
  playerRef.current.color = color;
}
```

스포이드 핵심

```
function pickColorFromCanvas(e) {
  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');
  const pos = getMousePos(canvas, e);
  const pixel = ctx.getImageData(pos.x, pos.y, 1, 1).data;

  const color = `rgb(${pixel[0]}, ${pixel[1]}, ${pixel[2]})`;
  changePlayerColor(color);
}
```

주의

- 스포이드는 배경을 그린 뒤의 Canvas 픽셀을 읽는다. 도형 위를 찍으면 도형 색을 찍을 수 있다.
- 처음에는 스포이드 모드일 때 Canvas 클릭이 “찾기 판정”이 아니라 “색 추출”이 되도록 분기한다.
- 숨는 사람 화면과 술래 화면의 클릭 역할이 다르다. 화면 모드에 따라 클릭 핸들러를 분리한다.

완료 기준

- 팔레트 색을 누르면 도형 색이 바뀐다.
- 스포이드 모드에서 배경을 누르면 도형 색이 배경색으로 바뀐다.
- 스포이드 모드 해제 후에는 일반 클릭/이동이 정상 동작한다.

12. 확대 편집 모드

목표

작은 도형 위에 직접 그리기 어렵기 때문에, 별도 모달에서 도형을 5배 확대하고 도형 내부에 그림을 그린다.

처음 구현 범위

- 모달을 연다.
- 260x260 편집 Canvas를 표시한다.
- 마우스로 선을 그린다.
- 적용 버튼을 누르면 drawingCanvas를 저장한다.
- 실제 도형에 그 그림을 반영한다.

ShapeEditorModal 기본 구조

```
function ShapeEditorModal({ color, onApply, onCancel }) {
  const editCanvasRef = useRef(null);
  const isDrawingRef = useRef(false);

  function startDraw(e) {
    isDrawingRef.current = true;
  }

  function stopDraw() {
    isDrawingRef.current = false;
  }

  function draw(e) {
    if (!isDrawingRef.current) return;
    const canvas = editCanvasRef.current;
    const ctx = canvas.getContext('2d');
    const pos = getMousePos(canvas, e);

    ctx.fillStyle = color;
    ctx.beginPath();
    ctx.arc(pos.x, pos.y, 4, 0, Math.PI * 2);
    ctx.fill();
  }

  return (
    <div className="modal">
      <canvas
        ref={editCanvasRef}
        width={260}
        height={260}
        onMouseDown={startDraw}
        onMouseUp={stopDraw}
        onMouseMove={draw}
      />
      <button onClick={() => onApply(editCanvasRef.current)}>□□</button>
      <button onClick={onCancel}>□□</button>
    </div>
  );
}
```

실제 도형에 그림 반영하는 개념

playerRef.current.drawingCanvas에 편집 Canvas를 복사해둔다. 실제 도형을 그릴 때 도형 모양으로 clip을 걸고 drawingCanvas 이미지를 도형 위치에 맞춰 그린다. 처음에는 정확한 마스크링이 어렵다면 네모/원부터 적용하고, 세모/별은 뒤로 미룬다.

완료 기준

- 확대 편집 모달이 열린다.
- 편집 Canvas에 그림을 그릴 수 있다.
- 적용하면 게임 화면의 도형 안에 그림이 보인다.
- 취소하면 이전 그림이 유지된다.

13. 이동 흔적, 노출 단서, 밸런스

목표

슬레 아이템이 없는 MVP0이므로, 슬레가 아무 단서 없이 찍는 게임이 되지 않도록 기본 단서를 넣는다.

단서	동작	구현 난이도	우선순위
이동 흔적	도형이 움직인 좌표를 2~3초간 흐리게 표시	중간	높음
감시 중 흔적 강화	watchedFloor와 player.floor가 같을 때 흔적이 더 오래 남음	중간	높음
층 이동 잔상	계단/엘리베이터 사용 위치에 짧은 잔상 표시	중간	중간
편집 중 윤곽 노출	확대 편집 중 도형 테두리를 잠깐 더 선명하게 표시	낮음	중간
클릭 피드백	오답 클릭 시 짧은 흔들림/반응	낮음	중간

이동 흔적 데이터 예시

```
const trailsRef = useRef([]);

function addTrail() {
  const p = playerRef.current;
  trailsRef.current.push({
    x: p.x,
    y: p.y,
    floor: p.floor,
    createdAt: Date.now(),
  });
}

function drawTrails(ctx, floor) {
  const now = Date.now();
  trailsRef.current = trailsRef.current.filter((trail) => now - trail.createdAt < 3000);

  trailsRef.current.forEach((trail) => {
    if (trail.floor !== floor) return;
    const age = now - trail.createdAt;
    const alpha = 1 - age / 3000;
    const fillStyle = `rgba(0, 0, 0, ${alpha * 0.15})`;
    ctx.fillStyle = fillStyle;
    ctx.beginPath();
    ctx.arc(trail.x, trail.y, 10, 0, Math.PI * 2);
    ctx.fill();
  });
}
```

밸런스 테스트값

항목	초기값	조정 기준
게임 시간	120초	슬레가 매번 못 찾으면 150초보다 클릭 수/단서를 먼저 조정
클릭 수	15회	막 클릭이 많으면 12회로 축소, 너무 어렵다면 18회로 확대
도형 크기	52px	너무 안 보이면 58px, 너무 쉽게 잡히면 48px
이동속도	160~180px/s	도망이 너무 쉬우면 150, 답답하면 190

14. 최종 작업 순서 체크리스트

이 순서를 지키면 된다. 체크박스를 하나씩 지우면서 진행한다.

단계	작업	완료 기준
1	프로젝트 세팅	Vite React JS 프로젝트 실행, 폴더 정리
2	화면 전환	Start -> ShapeSelect -> Game -> Result 이동
3	Canvas 루프	Canvas가 requestAnimationFrame으로 계속 렌더링
4	5층 배경	1F~5F 버튼 또는 상태 변경으로 배경 변경
5	도형 선택/렌더링	선택한 원/세모/네모/별이 Canvas에 표시
6	키보드 이동	도형이 WASD/방향키로 이동, 화면 밖 제한
7	계단 이동	계단 근처에서 E키로 인접 층 이동
8	술레 감시층	술레가 층 버튼으로 watchedFloor 변경
9	클릭 판정	같은 층에서 도형 클릭 시 술레 승리
10	타이머/클릭 수	120초, 15클릭, 승패 처리
11	조준 커서	술레 마우스를 따라다니는 조준 그래픽 표시
12	힌트박스	현재 위장 컨셉 + 이전 2개 표시
13	팔레트	도형 색 변경
14	스포이드	배경 픽셀 색 추출
15	확대 편집	260px 편집 Canvas에서 그림 그리기
16	도형 내부 그림	편집 그림이 실제 도형 안에 표시
17	이동 흔적/잔상	술레가 추적할 기본 단서 제공
18	UI 정리/밸런스	처음 보는 사람이 한 판 플레이 가능

멈췄을 때 판단 기준

- 기능이 안 되면 직전 단계로 돌아가라. 다음 단계로 억지로 넘어가지 않는다.
- Canvas가 안 보이면 게임 로직을 보지 말고 ref, width/height, CSS부터 확인한다.
- 이동이 안 되면 keydown/keyup 콘솔 로그부터 확인한다.
- 클릭 판정이 틀리면 getBoundingClientRect 좌표 변환부터 확인한다.
- 스포이드/그림 기능이 어려우면 일단 색상 팔레트까지만 붙이고 다음으로 넘어간다.

15. 초보자용 개념 정리

개념	이 프로젝트에서의 의미	사용 위치
useState	화면에 보여줘야 하는 값 저장. 바뀌면 React가 다시 그림.	screen, winner, clicksLeft, timeLeft
useRef	바뀌어도 React 리렌더가 필요 없는 값 저장.	player x/y, key 입력, mouse 좌표, trails
useEffect	컴포넌트가 화면에 나온 뒤 실행할 작업.	게임 루프 시작, 키보드 이벤트 등록, 타이머
Canvas	배경/도형/흔적/조준점 등을 직접 그리는 판.	GameScreen
requestAnimationFrame	브라우저 화면 갱신 타이밍에 맞춰 반복 실행.	게임 루프
HUD	게임 위에 표시되는 정보 UI.	시간, 클릭 수, 총 버튼, 힌트박스
Collision	클릭이나 이동이 특정 대상에 닿았는지 확인하는 판정.	도형 클릭, 계단 근처 판정

React와 Canvas 역할 분리 예시

기능	React	Canvas
시작 버튼	O	X
총 버튼	O	X
배경 그림	X	O
도형 이동	X	O
힌트박스 텍스트	O	X
조준 커서	X	O
결과 화면	O	X

참고 자료

- Vite 공식 가이드: <https://vite.dev/guide/>
- React 공식 문서: <https://react.dev/>
- React useRef 공식 문서: <https://react.dev/reference/react/useRef>
- React useEffect 공식 문서: <https://react.dev/reference/react/useEffect>
- MDN Canvas API: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

마지막 기준

완벽한 게임을 한 번에 만들려고 하지 말고, “오늘 실행되는 작은 기능 하나”를 완성한다. 이 프로젝트는 코드 양보다 순서가 더 중요하다.